MALIS Project 1: Multi-Armed Bandits

Final Report

Team Members: Téo Altmeyer - Thomas Decker - Soren Faucon

November 2026

Contributions

- **Thomas Decker:** Performed tasks 1, 5, 6 and wrote the report (tasks 5 and 6)
- **Téo Altmeyer:** performed tasks 2, 3 and wrote the report (tasks 2 and 3)
- **Soren Faucon:** performed tasks 4, 5 and wrote the report (tasks 4 and 5)

1 Task 2: Human Play Analysis and Strategy Formalization

1.1 Gameplay Comparison with Strategy Notes

Only one team member (Thomas Decker, fishing_log_decker.json) completed the Moodle activity before the deadline. Across all three seasons, his actions strictly matched his written strategy notes (Table 1):

Season	Days	Explored	Committed Spot	Raw Catch	Oracle	Exp. Ratio (η)
Season 1	30	Days 1-5	Spot 0 (Old Pier)	260.6 kg	300.0 kg	92.67%
Season 2	10	Days 1-5	Spot 0 (Old Pier)	71.5 kg	99.0 kg	80.10%
Season 3	30	Days 1-5	Spot 3 (Deep Channel)	270.4 kg	271.5 kg	92.78%

Table 1: Summary of human gameplay vs. oracle targets and noise-free efficiency (η).

- **Season 1 (30 days):** The player tested all 5 spots once during days 1-5 (yielding 9.0, 5.5, 1.8, 2.0, 0.0 kg) and exploited spot 0 exclusively for days 6-30, matching his note: "I tried all the fish spot and one had like the double amount [...] so I chose to always go there."
- **Season 2 (10 days):** He repeated the exact 5-day round-robin exploration (yielding 10.1, 1.0, 3.6, 0.2, 8.4 kg) and exploited spot 0 for days 6-10 ("Same strategy than previously").
- **Season 3 (30 days, shift at day 16):** After exploring spots on days 1-5 (2.2, 5.2, 1.5, 10.4, 3.0 kg), he committed to spot 3 for days 6-30 without ever re-exploring ("As always I did the same strategy").

1.2 Formal Algorithmic Rule (1-Sample Explore-Then-Commit)

For any bay with $K = 5$ spots over T days ($T \geq K$), let $A_t \in \{0, ..., K-1\}$ be the spot chosen on day t and R_t the observed catch:

1. **Exploration ($1 \leq t \leq K$):** Sample each spot once without replacement ($A_t = t - 1$) and record the initial catch $r_{A_t} = R_t$.

2. **Commitment ($K < t \leq T$):** Select the empirically best spot $a^* = \arg\max_a r_a$ (breaking ties randomly) and play $A_t = a^*$ for all remaining days.

2 Task 3: Separating Decisions from Luck & Season Comparison

2.1 Performance Measure Derivation

Raw total catch $R_{total} = \sum_{t=1}^{T} R_t$ conflates decision quality with two sources of luck in `fishing.py`: (1) bay richness, since spot means $\mu_t(a)$ are drawn uniformly in $[2.0, 10.0]$ kg, and (2) daily Gaussian noise, since $R_t = \max(0, \mu_t(A_t) + \sigma\epsilon_t)$ with $\sigma = 2.5$ kg. Using `bay.best_possible()` and `bay.reveal()`, where $\mu_t^* = \max_a \mu_t(a)$, we define two normalized metrics:

$$\rho_{empirical} = \frac{\sum_{t=1}^{T} R_t}{\sum_{t=1}^{T} \mu_t^*} \tag{1}$$

$$\eta_{expected} = \frac{\sum_{t=1}^{T} \mu_t(A_t)}{\sum_{t=1}^{T} \mu_t^*} = 1 - \frac{Regret_{expected}}{\sum_{t=1}^{T} \mu_t^*} \tag{2}$$

While $\rho_{empirical}$ removes bay richness disparity, $\eta_{expected}$ also eliminates daily Gaussian noise σ, measuring purely the expected value of the chosen actions. Running `RandomAgent` across 500 random Season 1 bays confirms that relative dispersion ($CV = SD/Mean$) drops from 19.18% (raw catch) to 15.96% ($\rho_{empirical}$) and 14.55% ($\eta_{expected}$), achieving a 24.1% variance reduction (Table 2).

Metric (500 Season 1 Bays, RandomAgent)	**Mean**	**SD**	**CV**
1. Raw Total Catch (R_{total})	180.3 kg	34.6 kg	19.18%
2. Empirical Performance Ratio ($\rho_{empirical}$)	0.696	0.111	15.96%
3. Expected Efficiency Ratio ($\eta_{expected}$)	0.694	0.101	14.55%

Table 2: Variance reduction across 500 random bays under a fixed decision policy.

2.2 Cross-Season Comparison

- **Shorter Horizon in Season 2 ($T = 10$):** Spending 5 days exploring consumed 50% of the season (vs. 16.7% in Season 1). Consequently, expected efficiency dropped from 92.67% to 80.10%, showing that static exploration is too costly on short horizons.
- **Environmental Shift in Season 3 ($t_{shift} = 16$):** True spot means changed at day 16. The player did not adapt at all, staying on spot 3 for all 25 exploitation days. Yet, he achieved $\eta = 92.78\%$ purely because spot 3 happened to remain the optimal arm after the shift.

3 Task 4: Bandit Algorithms Description

We implemented three agents in `agents.py`:

- **HumanRuleAgent (Soren):** It explores sequentially for 5 days, saves the best empirical mean, and commits to it for the remainder of the season.

- **EpsilonGreedyAgent (Thomas):** Follows Section 2.2, selecting a random action with probability ϵ and exploiting the best known empirical mean with probability $1 - \epsilon$. We introduced a constant step-size parameter α (Section 2.5) to track non-stationary problems (like for season 3).
- **UCBAgent (Soren):** Upper Confidence Bound selection (Section 2.7). It selects actions that maximize $Q(a) + c\sqrt{\frac{\ln t}{N_t}}$, inherently balancing exploitation (high $Q(a)$) and exploration (low N_t).

4 Task 5: Evaluation and Discussion

4.1 1000 Random Bays Comparison

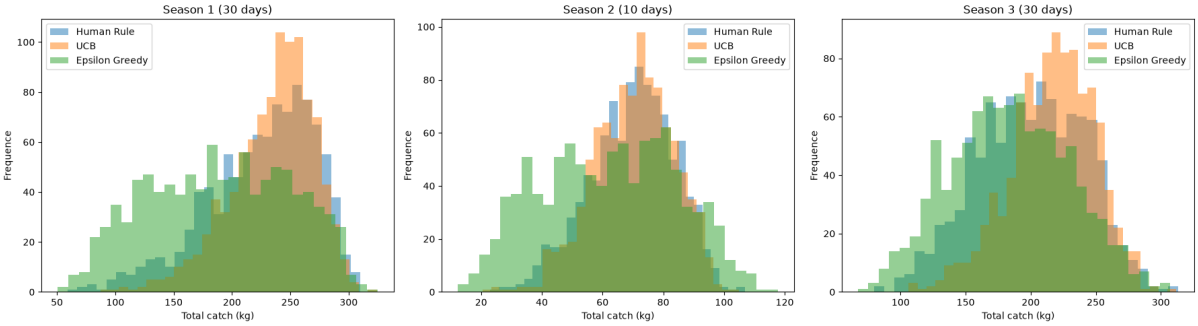


Figure 1: Distribution of total catch for each agent over 1000 random bays across seasons.

The catch distributions over 1000 random bays (Figure 1) demonstrate that `UCB` strictly dominates across all seasons, consistently exhibiting a right-shifted density peak compared to other agents. Conversely, the `EpsilonGreedy` distribution displays a prominent left-skewed failure tail.

4.2 Large-Scale Random Bays and Team Bays Comparison

We simulated `EpsilonGreedyAgent` ($\epsilon = 0.1, \alpha = 0.1$) and `UCBAgent` ($c = 2.0$). Evaluating the agents strictly on Decker's original game seeds (Table 3) produced seemingly miraculous performances for the Epsilon one on seasons 1 and 2. However, comparing this to a simulation over 200 random extended bays (Table 4) reveals the impact of sheer luck.

Agent	S1: 30 days	S2: 10 days	S3: 30 days, shift
Human Rule (Log)	260.6 kg	71.5 kg	270.4 kg
HumanRuleAgent	264.6 kg	80.4 kg	**275.8 kg**
UCB ($c = 2.0$)	262.6 kg	74.9 kg	269.9 kg
Eps-Greedy ($\epsilon = 0.1, \alpha = 0.1$)	**282.2** kg	**92.3 kg**	165.7 kg

Table 3: Agent comparisons over the specific bays played by our team.

4.3 Identifying the Best Agents for Seasons 2 and 3

- **Season 2:** On Decker's specific 10-day log (Table 3), `EpsilonGreedy` scored an incredible 92.3 kg. However, over 200 random 20-day bays (Table 4), it averages only 124.8 kg, heavily underperforming **UCB** (152.3 kg). This exposes a major algorithmic flaw: standard

Agent	Extended Season 2 (20d)	Extended Season 3 (100d)
Human (Log/Rule)	149.0 kg	687.0 kg
UCB ($c = 2.0$)	**152.3 kg**	**769.0 kg**
ϵ-Greedy ($\epsilon = 0.1, \alpha = 0.1$)	124.8 kg	614.4 kg

Table 4: Total Catch mean on 200 random extended bays.

`EpsilonGreedy` defaults to choosing Spot 0 on day 1. If Spot 0 yields a positive catch, it greedily exploits it for the rest of the short season, exploring other spots only 10% of the time. Decker's seed was a statistical "miracle" where Spot 0 happened to be optimal. Globally, **UCB** is the best short-term agent because its mathematical formulation forces an initial exploration of all arms, reliably finding the true optimum and safely beating `EpsilonGreedy`'s blind bias.

- **Season 3: UCB** is the most robust. On Decker's 30-day shift bay, the `HumanRule` won purely by luck because its initially chosen spot happened to remain optimal after the shift. When tested properly in a 100-day environment with a shift at day 50, `UCB` (769.0 kg) vastly outperformed `EpsilonGreedy` (614.4 kg). At a mere 10% exploration rate, `EpsilonGreedy` lacks the momentum to discover the new optimum and override its historic bias quickly enough, even with $\alpha = 0.1$.

4.4 Impact of modifying the parameter epsilon on the Epsilon Greedy Algorithm on season 1

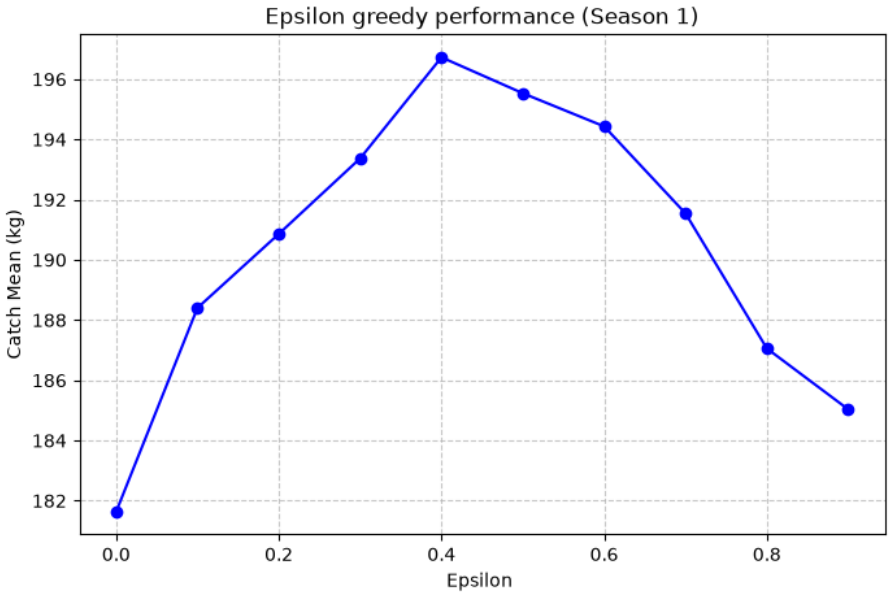


Figure 2: Impact of the exploration rate ϵ on the average total catch (Season 1 over 500 bays).

As shown in Figure 2, varying the exploration rate ϵ reveals an optimal performance peak around $\epsilon = 0.4$. A pure greedy approach ($\epsilon = 0.0$) performs poorly due to premature commitment to sub-optimal initial spots, while excessive exploration ($\epsilon > 0.5$) steadily degrades the total catch by wasting too many days on random pulls.

5 Task 6: Contextual Bandits

To leverage the wind, waves, and tide data provided in `ContextualBay`, we built a `KNNContextualAgent`. Because distances must not be disproportionately dominated by wind speed, we normalized the context space by the maximums found in `fishing.py`: `[40.0, 3.0, 2.0]`.

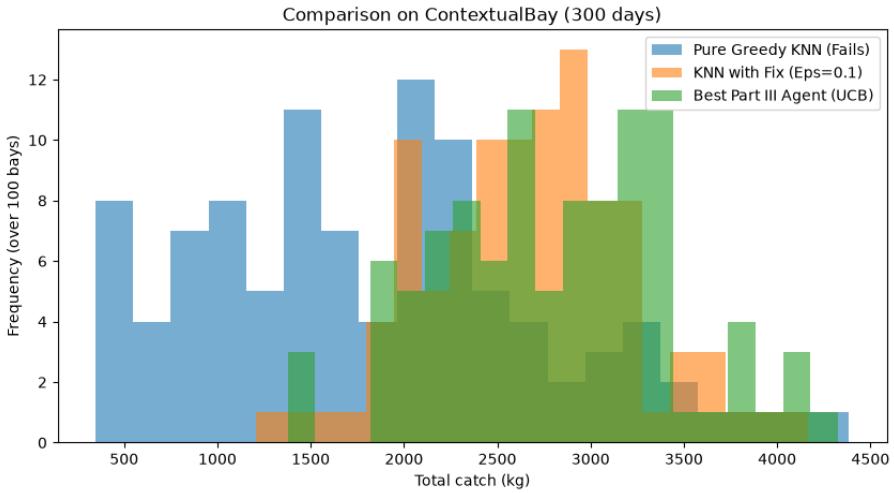


Figure 3: Comparison between pure K-NN, Epsilon Greedy K-NN and UCB algorithms

The Failure of Pure Greedy Prediction: When implementing pure K-NN prediction (Greedy, $\epsilon = 0.0$), the agent failed. It scored a mean of 1828.2 kg over 300 days—even worse than a purely Random agent (1945.7 kg). If the agent initially receives a poor reward (due to variance) on a spot under specific weather conditions, the K-NN predicts a low catch for similar future conditions. The agent then avoids that spot entirely, preventing it from ever updating its incorrect prediction.

The Fix and Comparison: To fix this, we introduced ϵ-greedy exploration ($\epsilon = 0.1$) on top of the K-NN. This forces the agent to sample untried spots, gathering robust data across all weather profiles. With this fix, the agent surged to 2664.0 kg, easily outperforming the naive predictor. While the `UCBAgent` scored slightly higher globally (2804.9 kg) due to the "ϵ-tax" (the fixed 10% exploration penalty the KNN incurs even when fully trained), the `KNNContextualAgent` proves that incorporating context drastically increases the ceiling of predictive accuracy over long horizons.

6 Use of AI

We used the AI for some parts of the project :

- **Data Visualization (1000 Random Bays Plot):** We used AI to generate the `matplotlib` code for the overlapping histograms. This ensured a clear, transparent, and aesthetically pleasing statistical comparison of the agents' performance distributions.
- **Code Architecture (UCB Wrapper):** AI assisted in designing the `UCBContextualWrapper` class. This allowed us to seamlessly adapt our existing Part III UCB agent to the Part IV contextual environment without needlessly duplicating the core algorithmic logic.
- **Algorithm Implementation (KNN ϵ-Greedy Fix):** We consulted the AI to understand how to correctly inject the ϵ-greedy exploration mechanism on top of the pure K-NN prediction loop within `agents.py` to resolve the local-minimum failure.

- **Document Formatting:** Finally, the AI was used to translate our raw text, draft notes, and Python data outputs from a Google Docs into clean, compilable LaTeX code to ensure a professional structure for this final report.